A Matheuristic for the Integrated Healthcare Timetabling Problem (IHTC-2024)*

Ahmed Elhefnawy
Weiyi Shen Qiaoyan Zeng
Advanced Modeling and Optimization, Group Project, TUM, Summer Term 2026

1 Task 1: Problem understanding and model

(a) The decision problem, in our own words. A hospital plans a timetable over a horizon of $D \in \{14, 21, 28\}$ days. Each day has three shifts: early, late, and night. The input describes patients waiting for surgery, occupants already in the hospital (with fixed rooms and stays), rooms, nurses, surgeons, and operating theatres (OTs). For each admitted patient the hospital chooses an admission day, a room that is kept for the whole stay, and an OT for the surgery, which takes place on the admission day. For every room that holds a patient in a given shift, the hospital assigns exactly one on-duty nurse. The IHTP, defined in the competition rules of Ceschia et al. (2025a), is the joint form of three classic subproblems that share the same beds, staff, and theatres: patient admission scheduling (PAS), surgical case planning (SCP), and nurse-to-room assignment (NRA). Some patients are mandatory and must be admitted inside a time window. Others are optional and may be postponed. A plan is feasible when it meets eight hard constraints (H1 to H8, listed in part (d)). Among feasible plans, the goal is to minimise a weighted sum of eight soft costs (S1 to S8, listed in part (e)).

(b) Sets, parameters, and decision variables. The tables below list the notation. The decision variables are the standard binary form of a scheduling model. We keep the room and the admission day in one variable (x) because a room is held for the whole stay, and we keep the theatre in a separate variable (y) because it is used only on the admission day.

Sets

$\mathcal{D} = \{0, \dots, D-1\}$, $\mathcal{S}$	days ($D \in \{14, 21, 28\}$) and the $3D$ shifts; shift s is on day $\lfloor s/3 \rfloor$
$\mathcal{P} = \mathcal{M} \cup \mathcal{P}^O$, $\mathcal{O}$	patients (mandatory $\mathcal{M}$, optional $\mathcal{P}^O$) and occupants
$\mathcal{R}, \mathcal{N}, \mathcal{U}, \mathcal{T}$	rooms, nurses, surgeons, theatres; $\mathcal{N}_s \subseteq \mathcal{N}$ on duty in shift s
$\mathcal{A}$	the ordered list of age groups
$\mathcal{R}_p$, $\mathcal{D}_p$	compatible rooms $\mathcal{R}_p \subseteq \mathcal{R}$ and legal admission days $\mathcal{D}_p = [\rho_p, \lambda_p]$ of p

Parameters

cap_r	beds in room r
$A_{t,d}$, $T^{\text{surg}}_{u,d}$	daily minute budgets of theatre t and surgeon u (0 = unavailable)
σ_n, $L_{n,s}$	nurse skill level and shift maximum load
ρ_p, λ_p, ℓ_p	release day, last admissible day (= due day if mandatory), length of stay
δ_p, u_p, g_p, age_p	surgery duration, surgeon, gender, age-group index of p
$w_{p,j}$, $\text{req}_{p,j}$	workload and required skill of p at stay-shift $j = 0, \dots, 3\ell_p - 1$
$W_1, \dots, W_8$	the eight soft-constraint weights

Decision variables (all binary)

$x_{p,d,r}$	patient p admitted on day d into room r (only for $d \in \mathcal{D}_p$, $r \in \mathcal{R}_p$)
z_p	optional patient p left unscheduled
$y_{p,d,t}$	surgery of p placed in theatre t on day d
$v_{n,r,s}$	nurse n covers room r in shift s (only for $n \in \mathcal{N}_s$)

*We pushed our entire solution into GitHub:
`https://github.com/ahmedelhefnawy21/ihtp-matheuristic-for-the-IHTC-2024-timetabling-problem`

(c) The model and the solution approach. A single MIP over all four variable families, with the nurses included, is too large to solve for these instances. We therefore split the problem into an upper layer and a lower layer. This is the standard decomposition for the IHTP, and Ceschia et al. (2025b) report it among the strong entries to the competition. The upper layer sets the admission day, room, and theatre (variables x, y, z). This fixes all hard constraints except nurse coverage, and it fixes the patient-side soft costs S1, S5, S6, S7, S8. The lower layer assigns nurses (variable v), which fixes the coverage constraint H8 and the nurse soft costs S2, S3, S4. We solve the two layers with the following pipeline. The pipeline pairs a heuristic search with exact mixed-integer programs, which is the matheuristic approach surveyed by Maniezzo et al. (2010). Each stage is present only because our ablation (§3) shows that removing it makes the result worse.

$$\text{construct} \to \underbrace{\text{PAS-MIP}}_{\text{admission}} \to \underbrace{\text{ALNS}}_{\text{refine}} \to \underbrace{\text{descent}}_{\text{true cost}} \to \underbrace{\text{CP-SAT LNS}}_{\text{capacity}} \to \underbrace{\text{OT-MIP}}_{\text{S5,S6}} \to \underbrace{\text{NRA-MIP}}_{\text{S2,S3,S4}}.$$

Construction. We build a first feasible plan quickly. Patients are inserted hardest first (mandatory before optional; then tighter admission window, longer surgery and stay first), each into its cheapest feasible day, room, and theatre. An optional patient is admitted only if its cost is below the penalty W_8 for leaving it out. A greedy order can box a mandatory patient out of a full surgeon schedule, so we repair H5 with ejection chains, the compound-move idea of Glover (1996) (we move the blocking patients to free capacity, with bounded backtracking), and, if needed, a short ruin-and-repair phase in the sense of the ruin-and-recreate principle of Schrimpf et al. (2000).

PAS-MIP (the admission core). This is the decisive stage, because the objective is dominated by unscheduled optional patients (S8), and admitting more of them is a packing problem that local search cannot solve well but a MIP can. Ceschia and Schaerf (2011) study patient admission scheduling and derive lower bounds for it by relaxation, and our admission model is the assignment form of that problem, specialised to this competition and matching the code that solves it. Using the admission variable $x_{p,d,r}$ (created only for feasible triples) and a per-room-day gender indicator $g_{r,\tau}$:

$$\begin{aligned}
\min\ & W_8 \sum_{p \notin \mathcal{M}} \Big(1 - \sum_{r,d} x_{p,d,r}\Big) + W_7 \sum_{p,r,d} (d - \rho_p)\, x_{p,d,r} \\
\text{s.t.}\ & \textstyle\sum_{r,d} x_{p,d,r} = 1 \ (p \in \mathcal{M}), \quad \le 1 \ (p \notin \mathcal{M}) && \text{(admit; H5, H6)} \\
& \textstyle\sum_{p \in A} x_{p,d,r}\, \mathbb{1}[p \text{ in } (r,\tau)] + \mathrm{occ}^A_{r,\tau} \le \mathrm{cap}_r (1 - g_{r,\tau}) && \text{(H1, H7, gender A)} \\
& \textstyle\sum_{p \in B} \cdots + \mathrm{occ}^B_{r,\tau} \le \mathrm{cap}_r\, g_{r,\tau} && \text{(H1, H7, gender B)} \\
& \textstyle\sum_{p:\, u_p = u} \delta_p x_{p,d,\cdot} \le T^{\mathrm{surg}}_{u,d}, \quad \sum_p \delta_p x_{p,d,\cdot} \le \sum_t A_{t,d} && \text{(H3, H4)}
\end{aligned}$$

The two gender constraints per room-day force every room-day to hold at most cap_r patients of one gender. Rooms and gender are handled exactly here. Theatre packing (S5, S6) is left to the OT-MIP, which keeps this model small.

ALNS (Adaptive Large Neighborhood Search). Ropke and Pisinger (2006) proposed adaptive large neighborhood search, which repeatedly removes a part of the plan and rebuilds it a better way. The removal operators (random, worst cost, empty a day, empty a theatre, and the related removal of Shaw (1998)) each target one cost term. The repair operators are greedy insertion, regret-2 insertion (which inserts next the patient that would lose the most by waiting, a standard repair in this framework), and randomised greedy. Moves are accepted by the simulated-annealing rule of Kirkpatrick et al. (1983), which accepts some worse plans early and tightens later. Feasibility is kept by construction, so the only violation the search can create is an unadmitted mandatory patient, which we handle with a large penalty. A short true-cost descent then makes single room or day moves, kept only when the full objective drops. It cleans the secondary costs (S1 age mix, and the nurse terms, since it re-scores the whole plan).

CP-SAT LNS (the capacity-creating step). The single PAS-MIP times out on the largest instances, and no small move can re-pack a full schedule to fit one more optional patient. We therefore free every still-unscheduled optional patient, plus everyone admitted inside a random

window of days, and re-pack that window exactly. This is a large neighborhood search, the destroy-and-rebuild scheme surveyed by Pisinger and Ropke (2010), with an exact re-pack in place of a heuristic one. The frozen patients' resource use is subtracted as residual capacity, so the small model stays valid. We solve it with CP-SAT, the constraint solver in OR-Tools, whose reified constraints model the gender, open-theatre, and surgeon-transfer indicators directly. Because admitting patients raises nurse cost, a step is kept only after the nurses are re-optimised and only if the full validated objective strictly improves. The search is therefore monotone. In our experiments this stage closes most of the residual admission gap.

OT-MIP. Operating-theatre scheduling is a research field in its own right, reviewed by Cardoen et al. (2010). With days and rooms fixed, each day is a small independent assignment problem: choose the theatre $y_{p,d,t}$ for each surgery to minimise open theatres (S5) and surgeon transfers (S6) subject to H4. Each day has few surgeries, so this solves to optimality in milliseconds.

NRA-MIP. Nurse-to-room assignment is a nurse rostering problem, the subject of the survey by Burke et al. (2004). With the layout fixed, the nurse variable $v_{n,r,s}$ places one on-duty nurse per occupied room-shift (H8). Overflow variables $o_{n,s} \geq 0$ model excess workload (S4), and the link $c_{e,n} \geq v_{n,r_e,s}$ counts distinct nurses per patient, so $\mathsf{S3} = \sum_{e,n} c_{e,n}$. The objective $W_2\mathsf{S2} + W_3\mathsf{S3} + W_4\mathsf{S4}$ is minimised over the whole horizon. The greedy nurse assignment is a feasible warm start, so this stage can only improve it. We give both of these models in full, with all their variables and constraints, in Appendix A, where the written form matches the code line for line; the description here covers their purpose and the costs they reduce.

(d) Hard constraints and feasibility. Feasibility rests on two things: the construction and the exact models keep every hard constraint, and the official validator confirms it. H2 (compatible rooms) and H6 (admission window) hold by construction, because we only ever create a variable $x_{p,d,r}$ for a legal day and a compatible room. H1 (no gender mix) and H7 (room capacity) are enforced exactly by the two gender constraints of the PAS-MIP, and are re-checked on every heuristic move. H3 (surgeon time) and H4 (theatre time) are enforced by the PAS-MIP and tracked per day during the search. H5 (all mandatory admitted) is secured by the construction, the ejection repair, and the equality constraint in the PAS-MIP. H8 (nurse presence: an on-duty nurse covers every occupied room-shift) is met by the greedy nurse assignment and the NRA-MIP, which choose $v_{n,r,s}$ only among nurses on duty in that shift. The official validator, our ground truth for feasibility, implements H8 as two checks: `UncoveredRoom` (every occupied room-shift has a nurse) and `NursePresence` (the assigned nurse is on duty in the shift). We never claim a solution feasible on our own evaluator alone: every reported solution is re-run through the official validator, and we report it as feasible only when it reports zero violations.

(e) Objective and soft constraints. We re-implemented the official objective in Python and pinned it to the validator with a golden test (§2), so the search optimises exactly what the competition measures. Each soft cost is reduced at the stage that controls it. Admission (PAS-MIP and CP-SAT LNS) reduces unscheduled patients (S8) and admission delay (S7). The OT-MIP reduces open theatres (S5) and surgeon transfers (S6). The NRA-MIP reduces under-skill (S2), continuity of care (S3), and excess workload (S4). The descent reduces the age mix (S1). The weight W_8 on unscheduled patients is by far the largest (150 to 500 across instances, against at most 50 for the others), so the objective is driven mostly by how many patients are admitted. This is why the admission MIP is the decisive stage.

2 Task 2: Implementation

(a) Python implementation. The solver is a Python package (`ihtp/`) with one module per concern: instance parsing, a solution model with incremental cost updates, a full objective evaluator, the constructive and ALNS heuristics, the four exact models, and an experiment harness.

The incremental model keeps running totals of cost and capacity, so testing one move costs time proportional to what the move touches, not to the whole instance. One command (`./run.sh`) builds the validator, runs the tests, solves all instances, computes the lower bounds, and writes every result.

(b) External solvers, libraries, and tools. We use **Gurobi 13.0.2** for the admission, theatre, and nurse MIPs and for the lower bound. Gurobi is the only commercial solver we use, and it is used under an academic license, which the assignment permits. We use the open-source **OR-Tools CP-SAT 9.15** for the capacity-creating search, and **NumPy 2.5** for the data arrays. Feasibility is checked by the official **IHTP_Validator** (IHTC, 2024) (the C++ program from the competition), which we compile from its source. No other solver is used.

(c) Reading instances and writing solutions. The program reads the official IHTC-2024 instance JSON files (IHTC, 2024) and writes solution JSON in the required schema (the admission day, room, and theatre per patient, and the nurse per occupied room-shift). The required results CSV (one line per instance, `i01,<objective>`, or `i01,infeasible`) is written by the harness.

(d) Validation. Our Python evaluator reproduces the official validator exactly on the published i01 reference solution (cost 3842, zero violations, and all eight cost components match), and we verified this on seven further instances that span the size range. This golden test means the search optimises the real objective.
Every solution we report is then re-checked by the official validator binary, and we keep the validator's own objective value as the reported number.

(e) Computational environment. The harness records the machine automatically (file `environment.txt`). The submitted run used an Apple M5 processor (10 cores), 34.4 GB of memory, and macOS 26.5.2 (arm64), with Python 3.12.13, Gurobi 13.0.2, OR-Tools 9.15.6755, and NumPy 2.5.1. The project sets no time limit. The full run (all 30 instances, five seeds each, all cores) took about 5.8 hours (20,953 seconds) on this machine. The per-instance runtime we report is the total solver time over the five seeds, which is the honest measure of the best-of-five effort. Runtime depends on hardware, so these numbers are stated for the machine above.

3 Task 3: Computational results and analysis

(a)-(c) Setup, feasibility, and objectives. We ran all 30 public instances. The method is feasible on every one (the official validator reports zero violations in all cases), so the feasibility column in Table 1 reads "yes" throughout. We compare each objective with the competition best-known value, which is the row minimum over the published results of the 32 teams reported by IHTC (2024). This best-known value is our benchmark. Every gap in this report measures our objective against it, so a positive gap means the field has published a better solution and a negative gap means we beat the field. The best-known baseline is a dated snapshot (6 July 2026) stored with the code, so our gaps stay fixed even if the competition site changes later. The metaheuristic is randomised, so we run five fixed seeds per instance and report the best of the five as the headline value, with the seed spread in Appendix B.

(d) Results table. Table 1 reports, for each instance, the feasibility, the objective, the best-known value, the gap, and the total runtime over the five seeds. The aggregate gap is 7.8%. We come within 0.5% of best-known on 2 instances and within 3% on 7. A group of mid-size and hard instances keeps double-digit gaps; the analysis below traces these to the nurse layer, not to admissions.

Table 1: Results on the 30 public instances. Best-known is the benchmark, and the gap is measured against it. Runtime is the total over five seeds.

instance	feasible	objective	best-known	gap%	runtime (s)	comment
i01	yes	4047	3842	5.34	988.9	
i02	yes	1444	1264	14.24	1182.6	
i03	yes	10605	10490	1.10	124.4	
i04	yes	2021	1884	7.27	6277.6	
i05	yes	12903	12760	1.12	5014.8	
i06	yes	10696	10671	0.23	2434.4	within 0.5% of best-known
i07	yes	5530	5026	10.03	6337.7	
i08	yes	6929	6291	10.14	6671.1	
i09	yes	7150	6682	7.00	5419.2	
i10	yes	21525	20820	3.39	5891.6	
i11	yes	25966	25938	0.11	5490.6	within 0.5% of best-known
i12	yes	13453	12430	8.23	5732.5	
i13	yes	17979	17328	3.76	6317.2	
i14	yes	10011	9746	2.72	6056.8	
i15	yes	13451	12486	7.73	6664.4	
i16	yes	11485	10139	13.28	6461.3	
i17	yes	47970	40535	18.34	7941.1	worst gap; nurse layer
i18	yes	37903	37660	0.65	5335.4	
i19	yes	49544	44587	11.12	12845.6	
i20	yes	30098	29098	3.44	6335.2	
i21	yes	28011	24703	13.39	9598.8	
i22	yes	53380	47861	11.53	11116.2	
i23	yes	39230	37550	4.47	12508.9	
i24	yes	34476	33221	3.78	10669.8	
i25	yes	12322	11517	6.99	8935.9	
i26	yes	71048	64613	9.96	8375.0	
i27	yes	62222	51828	20.05	10669.7	largest instance (493 patients)
i28	yes	76176	75172	1.34	6607.5	
i29	yes	14287	12475	14.53	10825.5	
i30	yes	40597	37943	6.99	10072.2	

All 30 feasible. Aggregate gap 7.8%, mean per-instance gap 7.4%. Within 0.5% of best-known on 2; within 3% on 7.

(e) Critical analysis: where the cost is, and what is hardest. We compared our solutions with the best-known solutions component by component, summed over all 30 instances. The result was not what we expected. In absolute size the objective is dominated by unscheduled patients (S8), but our admission stages drive S8 so far down that we schedule *more* optional patients than the whole field: our total S8 is 2% below best-known. The gap to best-known is therefore not in admissions at all. It is in the nurse layer: continuity of care (S3) is 39% of the total gap, under-skill (S2) is 30%, and excess workload (S4) is 22%, which is 90% together (Appendix C gives the full per-term breakdown). We diagnosed two causes, each with a controlled experiment of our own. First, the nurse MIP was starved of solver time: re-solving it with a larger work budget on the same fixed layout cut nurse cost by up to 16% on the hard instances, and by 0% on the small ones that were already optimal. This led us to the size-scaled nurse budget the pipeline now uses, which moved the aggregate gap from 9.2% to 7.8%. Second, a structural cause remains: the admission stage does not see the nurse cost that its choices create, so it admits patients who are expensive to staff. Our lower S8 is partly bought with higher nurse cost. This is the honest cost of decomposing a coupled problem. The clearest next steps are to price an estimate of nurse cost into the admission objective, and to loop the admission and nurse models rather than run

them once. Longer budgets and more seeds help only slightly (see the seed spread in Appendix B), which is consistent with the limit being the model coupling and not compute.

Ablation. * Table 2 adds the stages one at a time on representative instances. The construction is far from best-known. The PAS-MIP gives the single largest drop, because it fixes admissions. The ALNS and descent trim the secondary costs. The CP-SAT LNS admits the last hard optionals, and the exact OT and NRA models remove the final theatre and nurse cost. We dropped any component that did not help (for example, a best-fit packing rule we tried early on).

Table 2: Cumulative ablation on representative instances (seed 1, so the best-of-5 headline objective can be marginally lower). Each stage is kept because it lowers cost: the PAS MIP gives the largest drop (admissions); descent, the CP-SAT LNS and the exact OT/NRA polish each still trim cost.

instance	construct	+PAS	+ALNS	+descent	+LNS	+OT	+NRA	best-known
i04	2 833	2 505	2 188	2 188	2 022	2 022	2 021	1 884
i13	27 081	18 877	18 877	18 697	18 085	18 055	18 020	17 328
i16	–	12 127	12 127	12 043	11 533	11 533	11 532	10 139
i27	85 614	75 184	68 084	66 464	65 003	64 583	63 896	51 828

Certified optimality gap. The best-known values are upper bounds, not proven optima, so a gap to them cannot say how close we are to the true optimum. We therefore compute a valid lower bound of our own*. We solve a relaxation that keeps the admission decision and the two binding resources (surgeon-day and theatre-day time) with the patient-side costs they drive, and drops the room packing and the nurse layer, which only add non-negative cost. Dropping constraints and non-negative terms can only lower the optimum, so Gurobi's proven dual bound on this relaxation is a valid lower bound on the true optimum, which is a standard relaxation argument in integer programming (Wolsey, 1998). Ceschia and Schaerf (2011) compute relaxation-based lower bounds for patient admission scheduling in exactly this way. This gives a certified gap (Appendix D): the true optimum lies between the bound and our objective. The bound is loose, because it ignores the nurse and age-mix costs, so both our certified gap (57%) and the best-known's own certified gap (46%) are large and track each other across instances. This tells us the looseness is mostly in the bound. Adding the nurse terms to the bound is the natural way to make it sharp.

Seed robustness. Across the five seeds the objective is stable on the small instances (a standard deviation of a few units) and spreads on the hard ones (up to about 1300 on i17). Best of five improves on the average single seed by about one point of gap. The quality therefore comes from the method, not from running many restarts. The seeds serve two honest ends: a robustness estimate for a randomised search, and a reproducible way to recover the search breadth that our deterministic setup gives up (see the reproducibility note below).

Reproducibility. The method is deterministic. It uses no wall-clock stopping rule. The heuristic runs a fixed number of iterations, and each solver stops on a fixed amount of work (Gurobi's

*An ablation is a controlled experiment that measures what one part of a method contributes (Fawcett & Hoos, 2016). We build the pipeline one stage at a time, hold everything else fixed (the same instances, the same seeds, the same work budgets), and record the objective after each stage is added. A stage is kept only when its addition lowers the cost. This isolates the effect of each stage, so any improvement can be traced to the stage that caused it and not to chance or to some other change

*A certified optimality gap is the distance between our objective and a proven lower bound on the true optimum, given as a percentage of that bound (Wolsey, 1998). It differs from the gap to a best-known value, which only compares our result to another heuristic. The lower bound is guaranteed, so the true optimum cannot sit below it, and the real gap to the optimum is at most this figure. We obtain the bound by solving a relaxation of the problem to proven optimality (Appendix D)

WorkLimit, and a deterministic time budget for CP-SAT). For a fixed seed the solver returns the same solution on any machine that runs the same solver versions, and only the runtime changes with hardware. We do not claim bit-identical values across a different solver version or CPU architecture, since that is a property of the solver and not of our stopping rules; feasibility and the values are unaffected. The 30 instances are independent, so they run in parallel across all cores without breaking determinism.

4 Conclusion

A two-layer matheuristic solves the IHTP feasibly on all 30 public instances and comes within a fraction of a percent of the best-known value on the closest ones, at an aggregate gap of 7.8%. The design was driven by evidence throughout. We found the dominant cost (admissions), diagnosed its cause (capacity packing), and used the exact tool that fits it, keeping only the stages an ablation shows to help. Our component analysis then showed that the remaining gap is in the nurse layer, and we traced it to a solver budget that we fixed and to a model coupling that we describe as the next step. We consider the honest reporting of that residual, backed by our own experiments and a certified bound, to be as much a result as the objective values themselves.

References

Burke, E. K., De Causmaecker, P., Vanden Berghe, G., & Van Landeghem, H. (2004). The state of the art of nurse rostering. *Journal of Scheduling, 7*(6), 441-499.

Cardoen, B., Demeulemeester, E., & Beliën, J. (2010). Operating room planning and scheduling: A literature review. *European Journal of Operational Research, 201*(3), 921-932.

Ceschia, S., Rosati, R. M., Schaerf, A., Smet, P., Vanden Berghe, G., & Zanazzo, E. (2025a). *The Integrated Healthcare Timetabling Competition 2024: Problem description and rules*. Competition problem specification, last updated 10 January 2025. `https://ihtc2024.github.io`

Ceschia, S., Rosati, R. M., Schaerf, A., Smet, P., Vanden Berghe, G., & Zanazzo, E. (2025b). The Integrated Healthcare Timetabling competition 2024. *Operations Research, Data Analytics and Logistics, 45*, Article 200481. `https://doi.org/10.1016/j.ordal.2025.200481`

Ceschia, S., & Schaerf, A. (2011). Local search and lower bounds for the patient admission scheduling problem. *Computers & Operations Research, 38*(10), 1452-1463.

Fawcett, C., & Hoos, H. H. (2016). Analysing differences between algorithm configurations through ablation. *Journal of Heuristics, 22*(4), 431-458.

Glover, F. (1996). Ejection chains, reference structures and alternating path methods for traveling salesman problems. *Discrete Applied Mathematics, 65*(1-3), 223-253.

Integrated Healthcare Timetabling Competition (IHTC). (2024). *IHTC-2024: Instances, official validator, and published team results*. `https://ihtc2024.github.io`

Kirkpatrick, S., Gelatt, C. D., & Vecchi, M. P. (1983). Optimization by simulated annealing. *Science, 220*(4598), 671-680.

Maniezzo, V., Stützle, T., & Voß, S. (Eds.). (2010). *Matheuristics: Hybridizing metaheuristics and mathematical programming* (Annals of Information Systems, Vol. 10). Springer.

Pisinger, D., & Ropke, S. (2010). Large neighborhood search. In M. Gendreau & J.-Y. Potvin (Eds.), *Handbook of metaheuristics* (2nd ed., pp. 399-419). Springer.

Ropke, S., & Pisinger, D. (2006). An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows. *Transportation Science, 40*(4), 455-472.

Schrimpf, G., Schneider, J., Stamm-Wilbrandt, H., & Dueck, G. (2000). Record breaking optimization results using the ruin and recreate principle. *Journal of Computational Physics, 159*(2), 139-171.

Shaw, P. (1998). Using constraint programming and local search methods to solve vehicle routing problems. In *Principles and Practice of Constraint Programming (CP 1998)* (Lecture Notes in Computer Science, Vol. 1520, pp. 417-431). Springer.

Wolsey, L. A. (1998). *Integer programming*. Wiley-Interscience.

A The operating-theatre and nurse models

The report gives the admission model (PAS-MIP) and describes the last two exact models in words. The formulations below are our own, and they match our implementation in Python. They follow the standard model forms for operating-theatre scheduling (Cardoen et al., 2010) and nurse rostering (Burke et al., 2004), specialised to the costs and constraints of this competition.

OT-MIP, per day. For a fixed day d, the surgeries are the patients admitted on day d, and t ranges over the theatres open on day d. Variables: $y_{p,t}$ (surgery p in theatre t), ω_t (theatre t open), $g_{u,t}$ (surgeon u uses theatre t), and $\tau_u \geq 0$ (the number of extra theatres of surgeon u).

$$
\begin{aligned}
\min\ & W_5 \textstyle\sum_t \omega_t + W_6 \sum_u \tau_u \\
\text{s.t.}\ & \textstyle\sum_t y_{p,t} = 1 \quad \text{(each surgery } p\text{)} && \text{(one theatre)} \\
& \textstyle\sum_p \delta_p\, y_{p,t} \leq A_{t,d} && \text{(H4)} \\
& \omega_t \geq y_{p,t}, \qquad g_{u_p,t} \geq y_{p,t} && \text{(open / surgeon flags)} \\
& \tau_u \geq \textstyle\sum_t g_{u,t} - 1 && \text{(S6 transfers)}
\end{aligned}
$$

NRA-MIP. With the layout fixed, (r, s) ranges over occupied room-shifts. Variables: $v_{n,r,s}$ (nurse n, on duty in shift s, covers room r), $o_{n,s} \geq 0$ (workload of n in shift s above the cap), and $c_{e,n}$ (entity e, a patient or occupant, is seen by nurse n). Let $\text{gap}_{r,s,n} = \sum_{e \in (r,s)} \max(0, \text{req}_{e,s} - \sigma_n)$ be the skill shortfall, which is a constant, and let $w_{r,s}$ be the workload demanded in room r at shift s.

$$
\begin{aligned}
\min\ & W_2 \sum_{r,s,n} \text{gap}_{r,s,n}\, v_{n,r,s} + W_3 \sum_{e,n} c_{e,n} + W_4 \sum_{n,s} o_{n,s} \\
\text{s.t.}\ & \textstyle\sum_{n \in \mathcal{N}_s} v_{n,r,s} = 1 \quad \text{(each occupied } (r,s)\text{)} && \text{(H8)} \\
& o_{n,s} \geq \textstyle\sum_r w_{r,s}\, v_{n,r,s} - L_{n,s} && \text{(S4)} \\
& c_{e,n} \geq v_{n,r_e,s} \quad \text{(each shift } s \text{ of } e\text{'s stay)} && \text{(S3)}
\end{aligned}
$$

The greedy nurse assignment is a feasible warm start for both models.

B Seed distribution (five fixed seeds)

For each instance: the number of feasible seeds, the best, mean, worst, and standard deviation of the objective, and the best-of-five gap to best-known. Each seed is deterministic, so this table reproduces on the same solver versions.

inst	#seeds	best	mean	worst	std	best gap%
i01	5	4047	4084	4105	27	+5.3
i02	5	1444	1460	1474	12	+14.2
i03	5	10605	10627	10640	15	+1.1
i04	5	2021	2092	2181	59	+7.3
i05	5	12903	12936	13005	40	+1.1
i06	5	10696	10698	10700	2	+0.2
i07	5	5530	5697	5882	130	+10.0
i08	5	6929	6960	7034	43	+10.1
i09	5	7150	7163	7188	16	+7.0
i10	5	21525	21545	21550	11	+3.4
i11	5	25966	25984	26020	23	+0.1
i12	5	13453	13551	13763	122	+8.2
i13	5	17979	18014	18029	20	+3.8
i14	5	10011	10098	10157	55	+2.7
i15	5	13451	13789	14158	320	+7.7
i16	5	11485	11528	11575	34	+13.3
i17	5	47970	49596	51240	1288	+18.3
i18	5	37903	38041	38246	164	+0.6
i19	5	49544	49859	50055	260	+11.1
i20	5	30098	30110	30123	9	+3.4
i21	5	28011	28434	28768	287	+13.4
i22	5	53380	53546	53701	122	+11.5
i23	5	39230	39334	39405	81	+4.5
i24	5	34476	34539	34566	37	+3.8
i25	5	12322	12710	13398	428	+7.0
i26	5	71048	71343	72064	415	+10.0
i27	5	62222	63009	63963	856	+20.1
i28	5	76176	76204	76230	25	+1.3
i29	5	14287	14369	14535	96	+14.5
i30	5	40597	40743	40895	109	+7.0

Best-of-5: aggregate gap 7.8%. Mean per-instance gap 7.4% (best-of-5) vs 8.4% (average single seed); restarts recover 1.0 points.

C Component-level cost breakdown

Each soft term's total weighted cost, summed over the 30 instances, for our solutions and for the best-known solutions, with the difference and its share of the total gap. Our costs are read from the official validator logs, and the best-known costs come from running the same validator on the published reference solutions (IHTC, 2024). The nurse terms S2, S3, S4 make up about 90% of the gap, and we sit below best-known on S8, since we admit more patients.

soft term	our cost	best-known	difference	share of gap%
S1 age mix	7068	3251	+3817	+6.8
S2 nurse skill	38704	22197	+16507	+29.5
S3 continuity	123096	101331	+21765	+38.9
S4 workload	22004	9683	+12321	+22.0
S5 open OTs	33340	32680	+660	+1.2
S6 surgeon transfer	842	1318	-476	-0.9
S7 delay	137655	128150	+9505	+17.0
S8 unscheduled	409750	417950	-8200	-14.7
total	772459	716560	+55899	100

D Certified lower bounds

Our objective, the best-known value, the certified lower bound (Gurobi's dual bound on the relaxation of §3), and the resulting certified gaps. The relaxation solves to proven optimality on 6 of the 30 instances and stops at the deterministic work limit on the other 24. A work-limited stop still gives a valid lower bound, only a looser one.

inst	ours	best-known	LB	cert. gap%	BK cert. gap%
i01	4047	3842	3430	18.0	12.0
i02	1444	1264	655	120.5	93.0
i03	10605	10490	9710	9.2	8.0
i04	2021	1884	865	133.6	117.8
i05	12903	12760	12400	4.1	2.9
i06	10696	10671	10390	2.9	2.7
i07	5530	5026	1659	233.3	203.0
i08	6929	6291	3440	101.4	82.9
i09	7150	6682	2320	208.2	188.0
i10	21525	20820	14320	50.3	45.4
i11	25966	25938	25535	1.7	1.6
i12	13453	12430	6635	102.8	87.3
i13	17979	17328	7355	144.4	135.6
i14	10011	9746	5628	77.9	73.2
i15	13451	12486	6075	121.4	105.5
i16	11485	10139	5615	104.5	80.6
i17	47970	40535	20675	132.0	96.1
i18	37903	37660	36550	3.7	3.0
i19	49544	44587	26145	89.5	70.5
i20	30098	29098	21887	37.5	32.9
i21	28011	24703	10822	158.8	128.3
i22	53380	47861	29061	83.7	64.7
i23	39230	37550	27197	44.2	38.1
i24	34476	33221	30355	13.6	9.4
i25	12322	11517	4605	167.6	150.1
i26	71048	64613	43630	62.8	48.1
i27	62222	51828	29650	109.9	74.8
i28	76176	75172	67215	13.3	11.8
i29	14287	12475	6645	115.0	87.7
i30	40597	37943	21648	87.5	75.3

Aggregate certified gap 57.0% (ours) vs 45.6% (best-known). The bound is valid but loose (it drops the room, gender and nurse terms), so both gaps are large; the true optimum lies in [LB, ours].